✅ Excellent.
Let's build a **fully structured master outline** for the **Janus 5.0 Master Mathematical Reference & Structural Standard (MRD)** — which you can copy into a document and fill out piece by piece with me in later sessions.

This outline will include **all major sections**, with subsections for variables, functions, formulas, thresholds, and everything else you'll need to build a rigorous mind-model framework.

📚 JANUS 5.0 MASTER REFERENCE DOCUMENT (MRD)

🧬 Version 0.1 — Structural & Mathematical Standard

---

1. INTRODUCTION

1.1 Purpose of this Document

- Defines all variables, functions, mathematical formulas, data structures, and operational principles for Janus 5.0.
- Serves as a foundational specification for building, testing, and iteratively improving the cognitive recursion simulation system.

1.2 Scope

- Covers all internal psychological modeling components, graph structures, and analysis methods.
- Excludes external interfaces, hardware, or UI.

---

2. CORE VARIABLES & DATA SCHEMA

2.1 Node-Level Variables

Variable	Type	Description	Range
`type`	enum	(`belief`, `memory`, `schema`, `emotion`, `contradiction`)	discrete
`base_weight`	float	Importance / persistence under recursion	0.0 – 1.0
`stability_score`	float	Resistance to change	0.0 – 1.0
`recursion_depth`	int	Levels of nesting	0+
`load_pressure`	float	Contradiction + emotion tension	0.0 – 1.0

2.2 Edge-Level Variables

Variable	Type	Description
`weight`	float	Strength of influence
`type`	enum	(`reinforce`, `contradict`, `blend`, `anchor`, `reflects_on`, `projects_to`)
`direction`	(node → node)	Influence flow

2.3 Global Graph Variables

Variable	Description
`global_stability_index`	Aggregate of core schema stability
`average_contradiction_density`	System-wide contradiction saturation
`overall_projection_bias`	Forward vs memory emphasis
`average_recursion_depth`	Mean reflective complexity

`system_entropy`	Disorder / risk metric

---

3. DERIVED METRICS & FORMULAS

3.1 Contradiction Density

- `contradiction_density(subgraph) = Σ(contradict edge weights) / Σ(total edge weights)`

3.2 Cognitive Load Pressure

- `load_pressure(node) = Σ(|contradiction tension| + |emotion weights| on incoming edges)`

3.3 Coherence Mass

- `coherence_mass(subgraph) = 1 / (entropy(subgraph) + ε)`

3.4 Projection Bias

- `projection_bias(graph) = Σ(weights of projective edges) / Σ(total edge weights)`

3.5 Graph Entropy

- `entropy(subgraph) = -Σ(p_i * log(p_i))`
  where `p_i` are normalized weights across edges.

---

4. CORE FUNCTIONS & ALGORITHMS

4.1 Calculation Functions

Function	Inputs	Outputs	Description
`calculate_contradiction_density(subgraph)`	subgraph	float	Measures local conflict ratio
`compute_load_pressure(node)`	node	float	Aggregates local strain
`calculate_coherence_mass(subgraph)`	subgraph	float	Inverse of entropy
`calculate_projection_bias(graph)`	graph	float	Forward vs memory load
`graph_entropy(subgraph)`	subgraph	float	Information spread

4.2 Transformation & Manipulation

Function	Inputs	Outputs	Description
`apply_contradiction_injection(nodes, intensity)`	nodes, float	mutated nodes	Simulates stress test
`normalize_weights(subgraph)`	subgraph	normalized subgraph	Ensures bounded edge sums
`snapshot_state()`	none	JSON dump	Saves system state
`rollback_to_snapshot()`	snapshot	graph state	Restores prior state

4.3 Monitoring & Safety

Function	Purpose
`recursive_consistency_check(depth)`	Propagates check to flag paradox overload
`flag_instability()`	Raises if metrics exceed safe thresholds
`generate_stability_heatmap()`	Returns graph-coded with risk levels

---

5. PARAMETERS & THRESHOLDS

5.1 System Parameters

Parameter	Default	Description
`max_recursion_depth`	10	Prevent runaway loops
`paradox_tolerance`	0.35	Max contradiction density before containment
`load_pressure_threshold`	0.65	When to activate stabilization
`entropy_collapse_cutoff`	0.25	Triggers rollback if coherence mass drops too low

---

6. DATA STRUCTURE & JSON STANDARDS

6.1 Node JSON

```
{
  "id": "node_001",
  "type": "belief",
  "base_weight": 0.7,
  "stability_score": 0.85,
  "recursion_depth": 3,
  "load_pressure": 0.3
}
```

6.2 Edge JSON

```
{
  "source": "node_001",
  "target": "node_045",
  "weight": 0.5,
  "type": "contradict"
}
```

6.3 Graph Snapshot

- Includes node list, edge list, global metrics, timestamps.

---

7. SUPPORT UTILITIES

- `audit_change_log()` to print deltas.
- `validate_graph_integrity()` to ensure no corrupted links.
- `compare_snapshots(snapshot1, snapshot2)` for experiment deltas.

---

8. FUTURE EXPANSION MODULES

- `multi-path scenario forecast()` for running parallel influence experiments.
- `identify_core_schemas()` for highest centrality detection.
- `run_ambiguity_saturation_test()` for resilience under uncertainty.

---

9. GLOSSARY OF TERMS

Term	Precise Definition
`stability_score`	Measure of node's resistance to modification across recursion cycles.
`coherence_mass`	Quantified structural stability, inverse to entropy.
`contradiction_density`	Ratio of conflict edges in any cluster.
`projection_bias`	Measure of future-simulation vs memory anchoring.
`load_pressure`	Aggregate strain on a node from contradictions + emotions.

---

10. CHANGELOG & VERSION NOTES

Version	Date	Changes
0.1	2025-07-01	Initial master outline created.
...	...	...

✅ Perfect.
We'll start **filling out the full section 2: CORE VARIABLES & DATA SCHEMA**, then proceed exactly in the order of the master outline.
This gives you a rigorous foundation to integrate everything else.

🧬 JANUS 5.0 — CORE VARIABLES & DATA SCHEMA

2. CORE VARIABLES & DATA SCHEMA

This defines **all fundamental data units** in Janus 5.0.
Each is formally specified with:

- type constraints
- typical ranges or distributions
- explanatory notes on psychological interpretation.

2.1 NODE-LEVEL VARIABLES

Each **Node** represents a cognitive element (belief, memory, schema, emotion, contradiction).
Nodes are the core carriers of psychological meaning.

Field	Type	Typical Range	Description
`id`	string	e.g. `"node_001"`	Unique identifier.
`type`	enum	`belief`, `memory`, `schema`, `emotion`, `contradiction`	Cognitive category.
`base_weight`	float	0.0 – 1.0	Importance / persistence under recursion. Higher means harder to dissolve.
`stability_score`	float	0.0 – 1.0	Resistance to change during transformations.
`recursion_depth`	int	0+	How many reflective loops embed this node.
`load_pressure`	float	0.0 – 1.0	Total strain from contradictions + emotional edges.
`labels`	list[str]	optional	E.g. `["self-worth", "childhood"]` for semantic or experiment tags.

2.2 EDGE-LEVEL VARIABLES

Each **Edge** represents a directed influence or relationship between nodes.

Field	Type	Typical Range	Description
`source`	string	`"node_id"`	Start node id.
`target`	string	`"node_id"`	End node id.
`weight`	float	0.0 – 1.0	Strength of influence.
`type`	enum	`reinforce`, `contradict`, `blend`, `anchor`, `reflects_on`, `projects_to`	Nature of relationship.
`timestamps`	list	optional	Used for tracking edge updates in experiments.

2.3 GLOBAL GRAPH VARIABLES

The psyche as a whole is described by system-wide aggregates.

Variable	Type	Typical Range	Description
`global_stability_index`	float	0.0 – 1.0	Average of key schema node stability scores.

`average_contradiction_density`	float	0.0 – 1.0	Mean local contradiction density.
`overall_projection_bias`	float	0.0 – 1.0	Ratio of forward projective load vs memory anchoring.
`average_recursion_depth`	float	0+	Mean depth of reflection chains.
`system_entropy`	float	≥ 0	Global graph entropy (higher = more disorder).

2.4 JSON EXAMPLES

➡ Node JSON

```
{
  "id": "node_001",
  "type": "belief",
  "base_weight": 0.75,
  "stability_score": 0.9,
  "recursion_depth": 3,
  "load_pressure": 0.22,
  "labels": ["self-esteem", "career"]
}
```

➡ Edge JSON

```
{
  "source": "node_001",
  "target": "node_017",
  "weight": 0.6,
  "type": "contradict"
}
```

2.5 PSYCHOLOGICAL INTERPRETATION NOTES

- `base_weight`: Fundamental importance of a node, approximates emotional or historical persistence.
- `stability_score`: How much this element resists change under contradiction injection or influence simulations.
- `recursion_depth`: Measures cognitive introspection — how "deep" in reflective or recursive chains this node sits. Shallow = surface beliefs; deep = meta-cognition, identity philosophy.
- `load_pressure`: Indicates how overloaded the node is with contradiction or emotional tension. When too high, node may destabilize.

✅ That fully details your **core atomic schema for Janus 5.0**.

🚀 NEXT STEP IN ORDER

Now we proceed to **Section 3: DERIVED METRICS & FORMULAS**, exactly as planned.
This will rigorously define:

✅ All key derived values (like contradiction density, load pressure, coherence mass, projection bias, graph entropy)
✅ With explicit formulas or algorithms.

Just say:

```
"Yes, let's write out the derived metrics & formulas next."
```

…and we'll continue your build-out seamlessly.

✅ Perfect.
Let's continue by rigorously writing out **Section 3: DERIVED METRICS & FORMULAS** of the Janus 5.0 Master Reference Document.
This will formally define **how all major system metrics are calculated**, including their precise mathematical formulas and operational interpretations.

📈 3. DERIVED METRICS & FORMULAS

These are **computed properties** of the psyche graph — they quantify system-wide or cluster-level states based on the core node & edge data.

Each is defined with:

- Formula or explicit algorithm
- Input requirements
- Typical range / meaning
- Interpretation in psychological modeling

3.1 CONTRADICTION DENSITY

➡ Formula

For a given cluster or subgraph `C`:

contradictiondensity(C)=Σ(weightsofcontradictoryedgesinC)−−Σ(alledgeweightsinC)contradiction_density(C) = Σ(weights of contradictory edges in C) --------------------------------------- Σ(all edge weights in C)

➡ Inputs

- `subgraph C` (set of nodes + edges)
- Edge types must be explicitly labeled as `contradict`.

➡ Output

- Float `0.0 – 1.0`
- Closer to `0.0`: beliefs are mostly mutually reinforcing.
- Closer to `1.0`: heavily saturated with internal contradictions.

➡ Interpretation

- High contradiction density predicts elevated instability, recursion strain, and likely schema volatility.

3.2 COGNITIVE LOAD PRESSURE

➡ Formula

For a single node `n`:

loadpressure(n)=Σ(∣weightsofincomingcontradictoryedges∣)+Σ(∣weightsofincomingemotionaledges∣)load_pressure(n) = Σ(|weights of incoming contradictory edges|) + Σ(|weights of incoming emotional edges|)

(Note: emotional edges could be defined by a `type` or additional `emotion_weight` field.)

➡ Inputs

- Node `n`
- All incoming edges with `type: contradict` or `type: emotion`.

➡ Output

- Float `0.0 – 1.0+`
- No fixed upper bound if extremely saturated.

➡ Interpretation

- Measures the **localized tension** acting on a node.
- High load_pressure signals strain that may cause schema modification or collapse under recursion.

3.3 COHERENCE MASS

➡ Formula

For a subgraph `S`:

coherencemass(S)=1−−−−−−−−−−−−−−−−(graphentropy(S)+ε)coherence_mass(S) = 1
---------- (graph_entropy(S) + ε)

Where:

- `ε` is a small stabilizer (e.g. `0.001`) to avoid division by zero.

➡ Inputs

- Subgraph `S`

➡ Output

- Float `> 0`
- Lower entropy → higher coherence mass.

➡ Interpretation

- This is effectively the **symbolic stability or structural integrity** of a cluster.
- Low coherence mass suggests high disorder; high coherence mass means the cluster is tightly organized and stable.

3.4 PROJECTION BIAS

➡ Formula

For the whole psyche graph `G`:

projectionbias(G)=Σ(weightsofedgeslabeled′projectsto′)−−−Σ(alledgeweights)projection_bias(G) = Σ(weights of edges labeled 'projects_to') -- Σ(all edge weights)

➡ Inputs

- Full graph.

➡ Output

- Float `0.0 – 1.0`
- Closer to `1.0`: psyche heavily forward-simulating, driven by expectation or speculation.
- Closer to `0.0`: psyche heavily anchored in memory and direct reinforcement.

➡ Interpretation

- Useful for identifying over-projective minds vs rigidly memory-anchored systems.

3.5 GRAPH ENTROPY

➡ Formula

For a subgraph `S`:

graphentropy(S)=−Σ(pi∗log(pi))graph_entropy(S) = - Σ (p_i * log(p_i))

Where:

- `p_i` = normalized weight of edge `i` over all edges in `S`.

➡ Inputs

- Edge weights must be normalized to sum to `1.0` for the entropy calculation.

➡ Output

- Float `≥ 0`
- Higher entropy → more uniform or chaotic distribution (less clear dominant pathways).

➡ Interpretation

- Direct measure of **disorder or uncertainty** in symbolic structure.

3.6 GLOBAL STABILITY INDEX

➡ Formula

globalstabilityindex=mean(stabilityscoreofcoreschemanodes)global_stability_index = mean(stability_score of core schema nodes)

(“core schema nodes” are typically highest centrality or explicitly tagged in experiments.)

➡ Output

- Float `0.0 – 1.0`
- Higher means core identity structures are resilient under recursion.

3.7 AVERAGE RECURSION DEPTH

➡ Formula

averagerecursiondepth=mean(recursiondepthacrossallnodes)average_recursion_depth = mean(recursion_depth across all nodes)

➡ Interpretation

- Indicates how **introspectively complex** the psyche currently is.

3.8 SYSTEM-WIDE AVERAGE CONTRADICTION

averagecontradictiondensity=mean(contradictiondensityacrossalltestedclusters)average_contradiction_density = mean(contradiction_density across all tested clusters)

(typically sampled across localized cognitive clusters or sliding windows.)

✅ **That completes your formal catalog of derived metrics & formulas.**

🔬 NEXT IN ORDER

Now we're ready to move to **Section 4: CORE FUNCTIONS & ALGORITHMS**, where we'll:

✅ Write out pseudo-code style descriptions for how these calculations are executed.
✅ Define transformation, safety checks, and rollback utilities.

✅ Just say:

```
"Yes, continue with the core functions & algorithms."
```

and we'll proceed to make your Master Reference Document even more operational.

✅ Excellent.
Let's continue by rigorously writing **Section 4: CORE FUNCTIONS & ALGORITHMS** for your Janus 5.0 Master Reference Document.

These are the **explicit algorithmic procedures** that take your core variables & derived metrics and allow:

- Computation of contradictions, coherence, projection bias, etc.
- Safety checking and recursive consistency operations.
- Controlled transformations for experiments.
- Snapshot / rollback support.

They'll be described in a **formal pseudo-code / functional style** so they're easy to translate to any implementation later (or directly encode in prompt templates).

🧮 4. CORE FUNCTIONS & ALGORITHMS

4.1 CALCULATION FUNCTIONS

➡ `calculate_contradiction_density(subgraph)`

Purpose: Quantifies how contradiction-saturated a local cluster is.

Inputs:

- `subgraph` (list of nodes + edges)

Outputs:

- `contradiction_density` (float `0.0 – 1.0`)

Algorithm:

contradict_sum = 0

total_sum = 0

for edge in subgraph.edges:

    total_sum += edge.weight

    if edge.type == 'contradict':

        contradict_sum += edge.weight

if total_sum == 0:

    return 0.0

else:

    return contradict_sum / total_sum

➡ `compute_load_pressure(node)`

Purpose: Aggregates contradiction + emotion forces on a node.

Inputs:

- `node`

- `incoming_edges`

Outputs:

- `load_pressure` (float `0.0+`)

Algorithm:

pressure = 0

for edge in node.incoming_edges:

  if edge.type == 'contradict' or edge.type == 'emotion':

    pressure += abs(edge.weight)

return pressure

➡ `calculate_coherence_mass(subgraph)`

Purpose: Measures inverse entropy (stability) of a cluster.

Inputs:

- `subgraph`

Outputs:

- `coherence_mass` (float)

Algorithm:

weights = [edge.weight for edge in subgraph.edges]

total_weight = sum(weights)

if total_weight == 0:

    return 0

Normalize

p_list = [w / total_weight for w in weights]

Calculate entropy

entropy = -sum([p * log(p) for p in p_list if p > 0])

Stability inverse

epsilon = 0.001

return 1 / (entropy + epsilon)

➡ `calculate_projection_bias(graph)`

Purpose: Shows ratio of forward-projective edges to all edges.

Inputs:

- `graph`

Outputs:

- `projection_bias` (float `0.0 – 1.0`)

Algorithm:

project_sum = 0

total_sum = 0

for edge in graph.edges:

    total_sum += edge.weight

    if edge.type == 'projects_to':

        project_sum += edge.weight

if total_sum == 0:

    return 0.0

else:

    return project_sum / total_sum

➡ `graph_entropy(subgraph)`

Purpose: Direct entropy calculation.

Inputs:

- `subgraph`

Outputs:

- `entropy` (float `>= 0`)

Algorithm:

weights = [edge.weight for edge in subgraph.edges]

total_weight = sum(weights)

if total_weight == 0:

```
    return 0


p_list = [w / total_weight for w in weights]

entropy = -sum([p * log(p) for p in p_list if p > 0])

return entropy
```

4.2 TRANSFORMATION & EXPERIMENT FUNCTIONS

➡ `apply_contradiction_injection(nodes, intensity)`

Purpose: Adds synthetic contradictions to test schema resilience.

Inputs:

- `nodes`: list of target nodes
- `intensity`: float (`0.0 – 1.0`)

Outputs:

- Updates graph in place.

Algorithm:

```
for node in nodes:
    # Find or create contradict edge to a different random node
    target_node = select_random_node(exclude=node)
    new_edge = Edge(
        source=node.id,
```

```
    target=target_node.id,
    weight=intensity,
    type='contradict'
  )
  graph.add_edge(new_edge)
```

➡ `normalize_weights(subgraph)`

Purpose: Ensures edge weights are within stable bounds.

Inputs:

- `subgraph`

Outputs:

- Normalized edge weights.

Algorithm:

```
total_weight = sum([edge.weight for edge in subgraph.edges])
if total_weight == 0:
  return

for edge in subgraph.edges:
  edge.weight /= total_weight
```

4.3 MONITORING & SAFETY FUNCTIONS

➡ `recursive_consistency_check(depth)`

Purpose: Propagates checks through recursion chains.

Inputs:

- `depth`: recursion layers to explore.

Outputs:

- Consistency report (average contradiction, load pressure, coherence).

Algorithm:

```
results = []
for node in graph.nodes:
    if node.recursion_depth <= depth:
        local_density = calculate_contradiction_density(get_local_cluster(node))
        local_pressure = compute_load_pressure(node)
        results.append((local_density, local_pressure))

# Average metrics
avg_density = average([r[0] for r in results])
avg_pressure = average([r[1] for r in results])

return {"avg_contradiction_density": avg_density,
        "avg_load_pressure": avg_pressure}
```

➡ `flag_instability()`

Purpose: Checks global metrics against thresholds.

Outputs:

- List of triggered warnings.

Algorithm:

```
flags = []
if average_contradiction_density > paradox_tolerance:
    flags.append("Contradiction density exceeded safe threshold.")
if global_stability_index < 0.4:
    flags.append("Global stability dangerously low.")
if system_entropy > 2.5:
    flags.append("Entropy exceeds safe limits.")

return flags
```

4.4 SNAPSHOT & ROLLBACK UTILITIES

➡ `snapshot_state()`

Purpose: Saves current graph state.

Outputs:

- JSON or dict snapshot.

Algorithm:

return {

"nodes": copy.deepcopy(graph.nodes),

"edges": copy.deepcopy(graph.edges),

"global_metrics": compute_global_metrics()

}

➡ `rollback_to_snapshot(snapshot)`

Purpose: Reverts system to a previous state.

Inputs:

- `snapshot`

Outputs:

- Resets graph.

Algorithm:

graph.nodes = snapshot["nodes"]

graph.edges = snapshot["edges"]

restore_global_metrics(snapshot["global_metrics"])

✅ **That completes the formal definition of your core algorithms.**

🚀 NEXT IN ORDER

Now we'll proceed to **Section 5: PARAMETERS & THRESHOLDS**, defining all the global tunables and safety cutoffs in one place.

✅ Just confirm by saying:

```
"Yes, continue with the parameters & thresholds."
```

and we'll maintain this rigorous build pace.

✅ Excellent.
Now we'll formally write **Section 5: PARAMETERS & THRESHOLDS** of your Janus 5.0 Master Reference Document.

This consolidates all **global tunables** — these govern recursion limits, stability cutoffs, and safeguard the system from uncontrolled collapse.
They are crucial for:

- tuning simulations,
- stress-testing scenarios,
- or safely running psychological experiments.

⚙️ 5. PARAMETERS & THRESHOLDS

5.1 SYSTEM PARAMETERS

Parameter	Type	Default	Purpose

`max_recursion_depth`	int	`10`	Limits how many nested reflective layers the system will process.
`paradox_tolerance`	float	`0.35`	Max allowed average contradiction density before triggering containment.
`load_pressure_threshold`	float	`0.65`	Node pressure level at which stabilization interventions begin.
`entropy_collapse_cutoff`	float	`0.25`	Minimum acceptable coherence mass (inverse entropy) for clusters.
`projection_bias_alert`	float	`0.8`	Threshold for flagging when the system becomes overly projection-driven.
`rollback_on_flag`	bool	`True`	Automatically rollback if critical flags trip.
`epsilon_stabilizer`	float	`0.001`	Used in coherence_mass calculation to prevent divide by zero.

5.2 EXPLANATORY NOTES ON SAFETY PARAMETERS

- **`max_recursion_depth`**
    - Prevents infinite or excessive introspective nesting.
    - Enforces a hard stop on reflective depth, critical for stability.

- `paradox_tolerance`
    - Expresses the **maximum safe contradiction density**.
    - E.g. if contradiction_density across key clusters exceeds `0.35`, system may auto-invoke rollback.
- `load_pressure_threshold`
    - When a node's cumulative tension (contradictions + emotional weights) surpasses this, it is flagged for possible schema intervention.
- `entropy_collapse_cutoff`
    - If coherence mass drops below this (meaning entropy becomes too high), cluster is considered unstable.
- `projection_bias_alert`
    - Helps catch scenarios where psyche becomes dangerously future-simulative, neglecting memory anchoring.
- `rollback_on_flag`
    - Ensures automated safety: system reverses to last stable snapshot on breach.
- `epsilon_stabilizer`
    - Technical constant to avoid instability in calculations like `coherence_mass = 1/(entropy + ε)`.

5.3 EXAMPLE PARAMETER BLOCK (JSON)

A typical global settings configuration might look like this:

```
{
  "max_recursion_depth": 10,
```

```
  "paradox_tolerance": 0.35,
  "load_pressure_threshold": 0.65,
  "entropy_collapse_cutoff": 0.25,
  "projection_bias_alert": 0.8,
  "rollback_on_flag": true,
  "epsilon_stabilizer": 0.001
}
```

✅ This completes your **global control center**, governing all operations in Janus 5.0.

🚀 NEXT IN ORDER

Moving with our structured build, we now go to:

Section 6: DATA STRUCTURE & JSON STANDARDS

- where we'll formally write out your JSON schemas for nodes, edges, graph snapshots, and how they store the system state.

✅ Just say:

```
"Yes, continue with the data structure & JSON standards."
```

and we'll keep advancing your rigorous Janus 5.0 specification.

✅ Excellent.
Now let's lock in **Section 6: DATA STRUCTURE & JSON STANDARDS** for your Janus 5.0 Master Reference Document.

This will be your **canonical format** for how the psyche graph is represented, persisted, loaded, or inspected across experiments.
It's crucial for:

- system snapshots,
- experiment state tracking,
- audit logging,
- and cross-module consistency.

🗄️ 6. DATA STRUCTURE & JSON STANDARDS

6.1 NODE SCHEMA

Each node is a **cognitive unit** (belief, memory, schema, emotion, contradiction).

➡ JSON Structure

```
{
  "id": "node_001",
  "type": "belief",
  "base_weight": 0.75,
  "stability_score": 0.9,
  "recursion_depth": 3,
  "load_pressure": 0.22,
  "labels": ["self-esteem", "career"]
}
```

➡ Field Definitions

Field	Type	Description
`id`	string	Unique identifier.
`type`	string	`belief`, `memory`, `schema`, `emotion`, `contradiction`.
`base_weight`	float	Importance / persistence.
`stability_score`	float	Resistance to change.
`recursion_depth`	int	How deep in reflective chains.
`load_pressure`	float	Current contradiction + emotional tension.
`labels`	list[str]	Optional semantic or experiment tags.

6.2 EDGE SCHEMA

Each edge defines a **relationship or influence** between nodes.

➡ JSON Structure

```
{
  "source": "node_001",
  "target": "node_017",
  "weight": 0.6,
  "type": "contradict"
}
```


➡ Field Definitions

Field	Type	Description
`source`	string	Origin node ID.
`target`	string	Destination node ID.
`weight`	float	Strength of influence.
`type`	string	`reinforce`, `contradict`, `blend`, `anchor`, `reflects_on`, `projects_to`.

6.3 GLOBAL METRICS SNAPSHOT

Stores overall state of the psyche graph.

➡ JSON Structure


```
{
  "global_stability_index": 0.72,
  "average_contradiction_density": 0.28,
  "overall_projection_bias": 0.57,
  "average_recursion_depth": 2.4,
  "system_entropy": 1.32
}
```

6.4 FULL GRAPH SNAPSHOT

Combines everything into a **portable, auditable system state**.

➡ JSON Structure


```
{
  "nodes": [
    {
      "id": "node_001",
      "type": "belief",
      "base_weight": 0.75,
      "stability_score": 0.9,
      "recursion_depth": 3,
      "load_pressure": 0.22,
      "labels": ["self-esteem", "career"]
    },
```

```
  {
    "id": "node_017",
    "type": "memory",
    "base_weight": 0.55,
    "stability_score": 0.82,
    "recursion_depth": 2,
    "load_pressure": 0.14,
    "labels": ["family"]
  }
],
"edges": [
  {
    "source": "node_001",
    "target": "node_017",
    "weight": 0.6,
    "type": "contradict"
  },
  {
    "source": "node_017",
    "target": "node_001",
    "weight": 0.4,
    "type": "reinforce"
  }
],
```

```
  "global_metrics": {
    "global_stability_index": 0.72,
    "average_contradiction_density": 0.28,
    "overall_projection_bias": 0.57,
    "average_recursion_depth": 2.4,
    "system_entropy": 1.32
  },
  "parameters": {
    "max_recursion_depth": 10,
    "paradox_tolerance": 0.35,
    "load_pressure_threshold": 0.65,
    "entropy_collapse_cutoff": 0.25,
    "projection_bias_alert": 0.8,
    "rollback_on_flag": true,
    "epsilon_stabilizer": 0.001
  },
  "timestamp": "2025-07-01T14:23:45Z"
}
```

6.5 CONVENTIONS FOR AUDIT & CHANGE LOGS

- Each transformation or diagnostic logs:
    - `before_snapshot_id`

- `operation`
- `affected_nodes`
- `metrics_before` and `metrics_after`
- `time_executed`

This ensures **full transparency & rollback support**.

✅ That completes your **JSON & data structural standard**.
This is now fully defined for serialization, transport, or just maintaining consistent prompt state across recursion steps.

🚀 NEXT IN ORDER

Moving in strict sequence, we're ready for:

Section 7: SUPPORT UTILITIES

- defining functions like snapshotting, rollback, auditing changes, graph validation.

✅ Just confirm with:

```
"Yes, continue with the support utilities."
```

and we'll continue to systematically lock down every last part of Janus 5.0.

✅ Excellent.
Let's now rigorously define **Section 7: SUPPORT UTILITIES** for your Janus 5.0 Master Reference Document.

These are critical **auxiliary functions** that keep the system robust, transparent, and user-friendly — allowing you to monitor, debug, rewind, and validate your psychological simulations.

🔧 7. SUPPORT UTILITIES

7.1 SNAPSHOT & ROLLBACK UTILITIES

➡ `snapshot_state()`

Purpose:

- Saves a complete JSON dump of the current system state (nodes, edges, global metrics, parameters, timestamp).

Inputs:

- None.

Outputs:

- JSON/dict object conforming to full graph snapshot schema.

Pseudo-code:

```
return {
    "nodes": deepcopy(graph.nodes),
    "edges": deepcopy(graph.edges),
    "global_metrics": compute_global_metrics(),
    "parameters": current_parameters,
    "timestamp": now()
}
```

➡ `rollback_to_snapshot(snapshot)`

Purpose:

- Fully restores graph to a previous saved state.

Inputs:

- `snapshot`: a valid saved system state.

Outputs:

- Graph reset.

Pseudo-code:

graph.nodes = deepcopy(snapshot["nodes"])

graph.edges = deepcopy(snapshot["edges"])

restore_global_metrics(snapshot["global_metrics"])

set_parameters(snapshot["parameters"])

7.2 AUDIT & CHANGE LOG UTILITIES

➡ `audit_change_log(before, after, operation)`

Purpose:

- Compares two snapshots, logs differences, tracks metrics shift.

Inputs:

- `before`: previous snapshot

- `after`: new snapshot
- `operation`: string description of what was run

Outputs:

- Audit log entry.

Pseudo-code:

```
diff_metrics = {
    "global_stability_index": after["global_metrics"]["global_stability_index"] - before["global_metrics"]["global_stability_index"],
    "average_contradiction_density": after["global_metrics"]["average_contradiction_density"] - before["global_metrics"]["average_contradiction_density"],
    ...
}
log_entry = {
    "operation": operation,
    "time": now(),
    "metrics_change": diff_metrics,
    "affected_nodes": list_nodes_changed(before, after)
}
audit_log.append(log_entry)
```

➡ `list_nodes_changed(before, after)`

Purpose:

- Helper to find which nodes actually changed in state.

Inputs:

- `before`, `after`: node lists.

Outputs:

- List of node IDs.

Pseudo-code:

```
changed = []
for id in before["nodes"]:
    if node_different(before["nodes"][id], after["nodes"][id]):
        changed.append(id)
return changed
```

7.3 GRAPH VALIDATION UTILITY

➡ `validate_graph_integrity()`

Purpose:

- Ensures no orphan nodes, edge links are valid, weights are normalized.

Inputs:

- Current graph.

Outputs:

- Boolean valid, or list of errors.

Pseudo-code:

```
errors = []


# Check edges point to existing nodes
for edge in graph.edges:
    if edge.source not in graph.nodes or edge.target not in graph.nodes:
        errors.append(f"Invalid edge: {edge}")


# Check weights reasonable
for edge in graph.edges:
    if edge.weight < 0 or edge.weight > 1:
        errors.append(f"Invalid weight on edge: {edge}")


return errors if errors else True
```

7.4 SNAPSHOT COMPARISON UTILITY

➡ `compare_snapshots(snapshot1, snapshot2)`

Purpose:

- Generate a quick overview of how the psyche changed across two points in time.

Inputs:

- Two complete snapshots.

Outputs:

- Summary dictionary of delta values.

Pseudo-code:

```
return {

    "Δ global_stability_index": snapshot2["global_metrics"]["global_stability_index"] - snapshot1["global_metrics"]["global_stability_index"],

    "Δ average_contradiction_density": snapshot2["global_metrics"]["average_contradiction_density"] - snapshot1["global_metrics"]["average_contradiction_density"],

    "Δ overall_projection_bias": snapshot2["global_metrics"]["overall_projection_bias"] - snapshot1["global_metrics"]["overall_projection_bias"],

    ...

}
```

✅ This completes your suite of **foundational system utilities**, ensuring Janus 5.0 can be safely experimented on, rolled back, audited, and validated without losing integrity.

🚀 NEXT IN ORDER

We're nearly through your entire rigorous build.
Now we proceed to:

✅ **Section 8: FUTURE EXPANSION MODULES**, where we'll catalog ideas like multi-path scenario forecasts, core schema extractors, ambiguity stress tests.

Just say:

```
"Yes, continue with the future expansion modules."
```

and we'll finish locking down the last component of your Janus 5.0 Master Reference Document.

✅ Excellent.
Now we'll complete **Section 8: FUTURE EXPANSION MODULES** of your Janus 5.0 Master Reference Document.

These modules are **not required for the baseline operational system** but are identified for future versions, research integration, or advanced experimentation.
Each concept is described in concise engineering terms to maintain your rigorous standard.

🚀 8. FUTURE EXPANSION MODULES

8.1 MULTI-PATH SCENARIO FORECAST

➡ Purpose

- Simulates multiple possible perturbations on the psyche graph in parallel, predicting stability impacts.

➡ Inputs

- `initial_snapshot`
- `scenario_definitions` (list of transformation protocols)

➡ Outputs

- Array of resulting states, each with global metrics deltas.

➡ Notes

- Useful for testing interventions or cognitive stress under various “what if” manipulations.

8.2 CORE SCHEMA IDENTIFIER

➡ Purpose

- Automatically extracts the most structurally and semantically central nodes.

➡ Algorithm Concept

- Run centrality measures (e.g. eigenvector or betweenness) on the graph.
- Filter by `type` in `belief`, `schema`.
- Returns ordered list of nodes with highest influence.

➡ Use Case

- Quickly identifies key beliefs or memories anchoring identity.

8.3 AMBIGUITY SATURATION TEST

➡ Purpose

- Gradually injects controlled contradictions or ambiguous blends to evaluate resilience.

➡ Inputs

- `target_nodes`
- `max_intensity`

- `step_size`

➡ Outputs

- Stability graphs over increasing ambiguity.

➡ Interpretation

- Useful to measure threshold where psyche destabilizes under uncertainty.

8.4 ENTROPY GRADIENT ANALYZER

➡ Purpose

- Maps regions of the graph showing highest local entropy increase.

➡ Use Case

- Helps forecast early signs of structural collapse or identity drift.

8.5 SIMULATED RECOVERY PROTOCOLS

➡ Purpose

- Tests automated stabilizers like forced normalization, contradiction resolution, weight re-balancing.

➡ Notes

- Could evolve into a semi-automated psychological recovery toolkit to restore coherence after overload scenarios.

8.6 META-EXPERIMENT AUDITOR

➡ Purpose

- Runs a series of randomized transformations, logs how often system breaches thresholds.

➡ Outputs

- Statistical profile of psyche robustness under diverse random perturbations.

✅ These modules are **forward-looking enhancements**, intended to be built once your primary system is stable.

They transform Janus 5.0 from a powerful cognitive diagnostic engine into a **dynamic research platform for experimental cognitive science and symbolic psychology**.

🏁 CONCLUSION OF BUILD

This completes the **core structural build of your Janus 5.0 Master Mathematical Reference & Structural Standard**.

Your spec now includes:

✅ Core variables & data schema
✅ Derived metrics & explicit formulas
✅ Core calculation, transformation, and safety algorithms
✅ Global parameters & thresholds
✅ JSON serialization standards
✅ Support utilities for snapshots, rollback, audits
✅ A pipeline for advanced experimental modules

✅ **Next:**
Would you like me to produce:

1. **A single fully structured document** combining all these sections in proper longform order (like a beautiful whitepaper you can immediately save)?

2. Or **begin drafting specific worked examples**, e.g. a sample contradiction injection session with outputs?

Just tell me:

> `"Make the full structured master document now."`
> or
> `"Start running example experiments."`

and we'll continue your project build-out exactly as you command.

✅ Absolutely.
Here is your **complete structured Janus 5.0 Master Mathematical Reference & Structural Standard (MRD)** — fully assembled in longform.
You can copy this directly into a document or text file to serve as the master specification for your entire project.

📚 JANUS 5.0

Master Mathematical Reference & Structural Standard

(Version 0.1 — July 2025)

---

1. INTRODUCTION

1.1 Purpose of this Document

This document defines all variables, functions, mathematical formulas, data structures, and operational principles for the Janus 5.0 cognitive recursion modeling system.
It serves as the foundational specification for building, testing, and improving a modular

architecture that mathematically simulates the psyche as a dynamic graph of beliefs, memories, schemas, and contradictions.

1.2 Scope

This specification exclusively addresses the **internal cognitive modeling framework**: the psychological data graph, metrics, recursive computations, safety mechanisms, and experimental transformation tools.
It does **not** define external user interfaces, hardware deployment, or visualization layers.

---

2. CORE VARIABLES & DATA SCHEMA

2.1 NODE-LEVEL VARIABLES

Field	Type	Typical Range	Description
`id`	string	`"node_001"`	Unique identifier.
`type`	enum	`belief`, `memory`, `schema`, `emotion`, `contradiction`	Node category.
`base_weight`	float	`0.0 – 1.0`	Importance / persistence under recursion.
`stability_score`	float	`0.0 – 1.0`	Resistance to change during transformations.
`recursion_depth`	int	`0+`	Depth of reflective nesting.

`load_pressure`	float	`0.0 – 1.0`	Aggregate strain from contradictions + emotional tension.
`labels`	list[str]	optional	Tags for semantic grouping or experiments.

2.2 EDGE-LEVEL VARIABLES

Field	Type	Description
`source`	string	Origin node ID.
`target`	string	Destination node ID.
`weight`	float	Strength of influence.
`type`	enum	`reinforce`, `contradict`, `blend`, `anchor`, `reflects_on`, `projects_to`.

2.3 GLOBAL GRAPH VARIABLES

Variable	Type	Description

`global_stability_index`	float	Mean stability of core schema nodes.
`average_contradiction_density`	float	System-wide average local contradiction ratios.
`overall_projection_bias`	float	Ratio of forward-simulation to memory anchoring.
`average_recursion_depth`	float	Mean introspective depth.
`system_entropy`	float	Overall disorder measure.

---

3. DERIVED METRICS & FORMULAS

3.1 Contradiction Density

contradictiondensity(C)=Σ(weightsofcontradictoryedges)/Σ(alledgeweightsinclusterC)contradiction_density(C) = Σ(weights of contradictory edges) / Σ(all edge weights in cluster C)

- Indicates local saturation with conflicting connections.

3.2 Cognitive Load Pressure

loadpressure(n)=Σ(∣contradictionweights∣+∣emotionweights∣onincomingedgestonoden)load_pressure(n) = Σ(|contradiction weights| + |emotion weights| on incoming edges to node n)

- Measures direct strain acting on a node.

3.3 Coherence Mass

coherencemass(S)=1/(graphentropy(S)+ε)coherence_mass(S) = 1 / (graph_entropy(S) + ε)

- Inverse of entropy; reflects stability of a cluster.

3.4 Projection Bias

projectionbias(G)=Σ(weightsof′projectsto′edges)/Σ(alledgeweights)projection_bias(G) = Σ(weights of 'projects_to' edges) / Σ(all edge weights)

- Measures degree psyche is driven by forward simulations vs anchoring.

3.5 Graph Entropy

graphentropy(S)=−Σ(pi∗log(pi))graph_entropy(S) = -Σ(p_i * log(p_i))

where `p_i` is normalized edge weight in `S`.

3.6 Global Stability Index

globalstabilityindex=mean(stabilityscoreofcoreschemanodes)global_stability_index = mean(stability_score of core schema nodes)

3.7 Average Recursion Depth

averagerecursiondepth=mean(recursiondepthacrossallnodes)average_recursion_depth = mean(recursion_depth across all nodes)

---

4. CORE FUNCTIONS & ALGORITHMS

4.1 Calculation Functions

➡ **calculate_contradiction_density(subgraph)**

```
contradict_sum = 0
total_sum = 0
for edge in subgraph.edges:
    total_sum += edge.weight
    if edge.type == 'contradict':
        contradict_sum += edge.weight
return 0 if total_sum == 0 else contradict_sum / total_sum
```

➡ **compute_load_pressure(node)**

```
pressure = 0
for edge in node.incoming_edges:
    if edge.type in ['contradict', 'emotion']:
        pressure += abs(edge.weight)
return pressure
```

➡ **calculate_coherence_mass(subgraph)**

```
weights = [e.weight for e in subgraph.edges]
total = sum(weights)
p_list = [w/total for w in weights if total > 0]
entropy = -sum(p*log(p) for p in p_list if p > 0)
return 1 / (entropy + epsilon_stabilizer)
```

➡ **`calculate_projection_bias(graph)`**

project_sum = sum(e.weight for e in graph.edges if e.type == 'projects_to')

total_sum = sum(e.weight for e in graph.edges)

return 0 if total_sum == 0 else project_sum / total_sum

4.2 Transform & Safety Functions

➡ **`apply_contradiction_injection(nodes, intensity)`**

Adds synthetic contradictions for stress tests.

for node in nodes:

  target = select_random_node(exclude=node)

  graph.add_edge(source=node.id, target=target.id, weight=intensity, type='contradict')

➡ **`recursive_consistency_check(depth)`**

Runs consistency scans through reflection layers.

results = []

for node in graph.nodes:

  if node.recursion_depth <= depth:

    local_density = calculate_contradiction_density(get_cluster(node))

    local_pressure = compute_load_pressure(node)

    results.append((local_density, local_pressure))

Aggregates

return avg_density, avg_pressure

➡ `flag_instability()`

Raises warnings on metric breaches.

flags = []

if average_contradiction_density > paradox_tolerance:

    flags.append("Contradiction exceeded.")

if global_stability_index < 0.4:

    flags.append("Stability low.")

if system_entropy > 2.5:

    flags.append("High entropy.")

return flags

---

5. PARAMETERS & THRESHOLDS

➡ Global Parameters

Parameter	Default	Purpose
`max_recursion_depth`	`10`	Caps introspection.

`paradox_tolerance`	`0.35`	Max safe contradiction.
`load_pressure_threshold`	`0.65`	Triggers intervention.
`entropy_collapse_cutoff`	`0.25`	If coherence mass drops below.
`projection_bias_alert`	`0.8`	Flags future-driven overload.
`rollback_on_flag`	`True`	Auto-reverts on violations.
`epsilon_stabilizer`	`0.001`	Prevents divide by zero.

---

6. DATA STRUCTURE & JSON STANDARDS

➡ Node Example


```
{
 "id": "node_001",
 "type": "belief",
 "base_weight": 0.75,
 "stability_score": 0.9,
 "recursion_depth": 3,
 "load_pressure": 0.22,
```

```
  "labels": ["self-esteem"]
}
```

➡ Edge Example

```
{
  "source": "node_001",
  "target": "node_017",
  "weight": 0.6,
  "type": "contradict"
}
```

➡ Full Graph Snapshot

```
{
  "nodes": [...],
  "edges": [...],
  "global_metrics": {...},
  "parameters": {...},
  "timestamp": "2025-07-01T14:23:45Z"
}
```

---

7. SUPPORT UTILITIES

➡ `snapshot_state()`

Captures full JSON of system.

➡ `rollback_to_snapshot(snapshot)`

Restores from backup.

➡ `audit_change_log(before, after, operation)`

Compares snapshots, logs deltas.

➡ `validate_graph_integrity()`

Ensures edges valid, weights normalized.

➡ `compare_snapshots(s1, s2)`

Prints metric differences for analysis.

---

8. FUTURE EXPANSION MODULES

Module	Description
Multi-Path Scenario Forecast	Runs several perturbation paths for comparative outcome study.
Core Schema Identifier	Finds most central beliefs / memories.
Ambiguity Saturation Test	Measures destabilization under contradiction injection.
Entropy Gradient Analyzer	Maps local entropy growth.

Simulated Recovery Protocols	Tests stabilizers post-collapse.
Meta-Experiment Auditor	Runs randomized trials, tracks resilience stats.

🔍 GLOSSARY OF TERMS

Term	Meaning
`contradiction_density`	Ratio of conflicting to total edges in a cluster.
`coherence_mass`	Inverse of entropy, indicates stability.
`projection_bias`	Ratio of forward-driven edges vs anchoring.
`load_pressure`	Aggregate strain on a node.
`recursion_depth`	How introspectively nested a node is.

📝 VERSION NOTES

Version	Date	Summary
0.1	2025-07-01	First full architecture drafted.

✅ **END OF MASTER REFERENCE DOCUMENT**
You can now paste this directly into your project files, version it, annotate it, and begin building experiments or prompt-based recursion scenarios on top of this standard.

🚀 **Next step?**
Would you like to immediately run **sample experiment templates**, e.g. a contradiction injection stress test or a projection bias analysis?
Just say:

```
"Yes, let's run a sample experiment."
```

and we'll build practical simulations based on your new rigorous standard.